

Development Plan CADENCE

Team #4, CADENCE

Anam Khan
Haniya Kashif
Vivian Nguyen
Kelsey Rough
Hareem Salman

Table 1: Revision History

Date	Developer(s)	Change
September 25, 2026	Name(s)	Description of changes
...	...	...

[Put your introductory blurb here. Often the blurb is a brief roadmap of what is contained in the report. —SS]

[Additional information on the development plan can be found in the [lecture slides](#). —SS]

1 Confidential Information?

[State whether your project has confidential information from industry, or not. If there is confidential information, point to the agreement you have in place. —SS]

[For most teams this section will just state that there is no confidential information to protect. —SS]

2 IP to Protect

[State whether there is IP to protect. If there is, point to the agreement. All students who are working on a project that requires an IP agreement are also required to sign the “Intellectual Property Guide Acknowledgement.” —SS]

3 Copyright License

[What copyright license is your team adopting. Point to the license in your repo. —SS]

4 Team Meeting Plan

[This section is for meetings between the team members, and meetings between the team and their supervisor/advisor (as appropriate). —SS]

[How often will the team meet? where? when? —SS]

[If the meeting is a physical location (not virtual), out of an abundance of caution for safety reasons you shouldn’t put the location online —SS]

[To help schedule meetings, you can use the lecture/tutorial time when we do not have anything else scheduled. —SS]

[How often will you meet with your supervisor/advisor? when? where? —SS]

[Will meetings be virtual? At least some meetings should likely be in-person. —SS]

[How will the meetings be structured? There should be a chair for all meetings. There should be an agenda for all meetings. —SS]

5 Team Communication Plan

[Issues on GitHub should be part of your communication plan. —SS]

6 Team Member Roles

Team Lead: Coordinates overall project progress and deadlines, tracks blockers and dependencies, and ensures that responsibilities are distributed fairly among team members.

Meeting Chair: Organizes and leads team meetings, prepares the agenda, keeps discussions focused, and ensures that decisions and action items are identified.

Notetaker: Records meeting minutes, decisions, and action items, and keeps track of questions or issues that need to be brought to the supervisors.

Team Liaison: Acts as the team’s primary point of contact with the capstone professor and TAs. This member will communicate course-related questions and messages to and from the rest of the team.

Supervisor Liaison: Coordinates communication with the project supervisors; this includes organizing supervisor meetings, collecting questions from the team, and communicating responses back to the team.

Research Lead: Organizes useful research papers, technical references, and other relevant resources. The Research Lead will make sure these resources are accessible to the entire team.

Reviewer: Conducts final review of deliverables before submissions.

All team members will contribute to development, testing, research, and documentation. Roles will be reviewed after each major deadline and may be reassigned as the team’s needs change. The Meeting Chair and Notetaker will rotate every meeting.

7 Workflow Plan

The team will use Git and GitHub to manage source code, track issues, coordinate development, and house documentation for submission. The main branch will contain code that has been reviewed and is ready for integration. Team members will work on separate feature or bug-fix branches rather than making changes directly to main. Pull requests will be used to merge changes into main. Each pull request should describe the changes made, reference the relevant issue when applicable, and include information about testing performed. At least one other team member will review each pull request before it is merged.

Issues will be used to track development tasks, bugs, and other work that needs to be completed. Issues will include a clear description of the problem or task and, where appropriate, acceptance criteria or expected outcomes. Labels will be used to classify issues, such as feature, bug, hardware, testing, and documentation. Issues will be assigned to team members and used to track

work throughout the project. Larger tasks may be divided into smaller issues so that progress can be monitored more easily.

The team will use GitHub Actions for continuous integration. Pull requests will automatically run linting, unit tests, and regression tests where applicable. Required checks will be configured so that pull requests that do not meet the team's testing and quality standards cannot be merged into the main branch. Continuous delivery will not be a major part of the project because the final system is intended to run on dedicated hardware rather than being deployed to a production server.

8 Project Decomposition and Scheduling

The team will use GitHub Projects to organize and track the work required to complete the project. Issues will be created for tasks. The project kanban board will be organized by status, such as backlog, ready, in progress, in review, and done. Issues will be assigned to team members and associated with the relevant project stage. The board will be reviewed during team meetings to monitor progress, identify blocked tasks, and adjust priorities as needed.

The team's GitHub Repository can be accessed at: <https://github.com/Kelsey379/CADENCE>

The team's GitHub Project can be accessed at: <https://github.com/users/Kelsey379/projects/3>

Course Week	Major Project Stage	Deliverable
Week 3	Project setup and initial research	Team Formed, Project Selected
Weeks 3–6	Research, system investigation, hazards, and requirements	Problem Statement, POC Plan, Development Plan; Requirements Document and Hazard Analysis Rev. 0
Weeks 5–8	Signal acquisition and characterization; begin POC development	V&V Plan Rev. 0
Weeks 7–12	Proof of concept and initial signal-processing pipeline	Design Document Rev. -1, CD-Fall Rev. 0; POC Demonstration and Interviews
Weeks 10–16	POC validation and system design refinement	Design Document Rev. 0; Revision 0 Demo and Interviews
Weeks 14–19	Real-time implementation, hardware integration, and validation	V&V Report Rev. 0, CD-Winter Rev. 0
Weeks 19–22	Final system testing and refinement	Final Demonstration (Revision 1)
Weeks 22–24	Final system preparation and demonstration	EXPO Demonstration
Weeks 24–26	Final validation, documentation, and handoff	Final Documentation (Revision 1), including source code

Table 2: High-level project schedule aligned with course deadlines

The project stages are intentionally overlapping rather than sequential. Research, development, testing, and documentation will continue throughout the project as new information becomes available. The team will use GitHub Projects to decompose each stage into smaller tasks and track their progress. The schedule will be reviewed after each major course milestone and adjusted based on progress, hardware constraints, and supervisor feedback.

9 Proof of Concept Demonstration Plan

Based on our project’s scope and requirements, we have identified several potential risks that we plan to address. These include:

- **Signal acquisition:** The system relies on us being able to accurately capture and interpret the Doppler ultrasound signal that the Vivid Q ultrasound machine outputs. We currently expect this to be a pair of quadrature-demodulated channels, which must be sampled simultaneously to identify flow directions and velocities. If we are unable to get a clean signal, or if there are unexpected voltage and noise levels, then we may

need to add some additional processing before being able to process and interpret it as intended. This would have an impact on all further stages of the project.

- **Mean Blood Velocity (MBV) estimation:** The MBV estimation process has several steps. These include combining the I and Q channels into a single complex signal, filtering out low-frequency, high-amplitude artifacts from vessel wall motion, and then processing using Fourier transforms to split the signal into forward and reverse flows. We will then need to estimate frequency shifts from the signal spectrum and calculate the velocities from there. With this, there are risks of noise, filter settings, aliasing, or spectral broadening, which could occur at any phase and would make our velocity output inaccurate. Because of this, we will be looking into several methods of doing these steps and will be prototyping before implementation to ensure that we have an accurate way of doing all of this.
- **Real-time performance and hardware:** Because the system needs to work in real-time, we will need to make sure that we account for latency in our signal processing and the hardware we use. For a POC demo we expect to be running on a computer with recorded and simulated test data rather than low-level real-time hardware, as we will still be prototyping, but this is something we will need to account for in our design for the final system.
- **Output validation:** Because the output of this system is going to be used for medical research, we need to ensure that it is accurate, both in measurement and in timing. The lab's calibration procedures rely on this signal, so we will need to test thoroughly against their old test and calibration data to make sure that our solution is both precise and accurate.

These risks may change as we learn more about the equipment and the signal. Our proof of concept will demonstrate a minimal pipeline running on a computer, which will go from the Doppler I/Q input to an output voltage signal. This will allow us to validate our signal processing approach, while leaving the implementation of low-level hardware development and real-time digital signal processing for later in the development process. This proof of concept will confirm that our signal processing steps are accurate and produce the desired output, and we will ensure that this is correct before moving on to the real-time and hardware aspects.

10 Expected Technology

Our technology choices listed below are based on what we currently know about the project, and we may see them change as we learn more about the signal and equipment in the following weeks. Our plan is to develop this system in two stages. The first will be some high-level processing to ensure that our approach

is correct and to develop our proof of concept. The next stage will involve moving the validated process on to hardware to run in real-time. Below are the technologies we expect to use during both of these stages:

- **Programming Languages:** We plan to use Python for our initial prototyping, analysis, and proof of concept, as it allows us to easily test different signal processing methods and procedures, and visualize our results. The final product, which runs low-level on its own hardware, will be written in C or C++, since microcontrollers like the Teensy cannot run Python and we need efficient code to process the signal in real-time, consistent with the project requirements.
- **Libraries:** During the POC phase, we expect to use a number of Python libraries such as NumPy and SciPy for signal processing, filtering, and transforms. Matplotlib will be used to visualize our traces and signals before, during, and after each step of the signal processing. For the real-time system, we will use whichever libraries are compatible with the hardware we select. One current proposed idea is to use a Teensy 4.1 board with an external DAC. In this case, we would use the Teensy Audio Library to capture the stereo input, along with ARM's CMSIS-DSP library for the complex transforms and filtering.
- **Hardware:** We expect to use some analog circuitry to connect to the Doppler input and filter out interference before it is digitized, as well as a commercial off-the-shelf board that supports DSP, such as a Teensy 4.1. With this, we may use an external digital-to-analog converter (DAC), as well as adapters to output to the physical wired connections that the PowerLab 8/30 device takes as input for LabChart. Further details of the hardware we will use are still to be determined, as we are currently exploring our options.
- **Linting and Formatting:** We plan to use Pylint to lint our code during the proof of concept phase, and Black for Python formatting. Later on, in the final development stage when we shift to C or C++, we expect to use Clang-Format, Clang-Tidy, and Cppcheck for our linting and formatting.
- **Testing:** We will use Pytest for our Python development and a framework like Unity for C/C++ testing. Because our project also involves the use of hardware, we will use LabChart output and an oscilloscope to verify the final system output.
- **Continuous Integration:** We will use GitHub Actions to run the linter and unit tests on every pull request. Since code that depends on physical hardware cannot run in CI, we will keep the processing logic separate from the hardware input and output code.
- **Version Control:** We will be using Git and GitHub for version control, along with GitHub Issues and GitHub Projects for task tracking.

- **Development Environment:** We will use Visual Studio Code as our main IDE for development as well as for our L^AT_EX documentation.
- **Laboratory Equipment:** We will be using the Vascular Dynamics Lab's hardware for some testing and integration, which includes a GE Vivid Q ultrasound, a Multigon 500M Transcranial Doppler, a PowerLab 8/30 unit, and the LabChart software for signal input, recording, comparison, and validation.
- **Components Implemented Ourselves:** While we will be using libraries for standard building blocks like Fourier transforms and filters in both Python and C/C++, we plan to implement project-specific parts of the signal processing ourselves. This may include arranging the input transform and separating forward and reverse flow, designing the initial filter for the lab's measurements, estimating the mean frequency from the spectrum, converting the Doppler shift to velocity, and calibrating the final output.

11 Use of AI and LLMs

[This section is a continuation of the previous section. The role of LLMs in your project as important enough to warrant their own section. How will your team be using LLMs for code and documentation development? What tools will you be using? How will you verify the output of your LLMs? —SS]

12 Coding Standard

[What coding standard will you adopt? —SS]

13 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

[This section is about the use of AI to write this document, not your planned use of AI for your project. You discuss that topic in one of the sections in the body of this report. —SS]

13.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

13.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

13.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

Appendix — Reflection

1. Why is it important to create a development plan prior to starting the project?

Creating the development plan before we start the project is essential for this project, as we're currently working with a lot of unknowns and with some unfamiliar concepts and domain knowledge. Creating a detailed development plan forces us to think through the problem fully and break it down into smaller and more manageable steps, while getting a better understanding of how all of the steps and stages flow into one another. By developing this plan we were able to identify some gaps in our knowledge and find what we need to ask our supervisors about, read into, and learn for ourselves. For example, we all had to look into impedance matching for our ADC and DAC interfaces so that we would be able to accurately deal with the signals.

Creating the plan also helped us decide what order to do things in. At first we had been debating going straight to work on hardware, but when we looked at all there was to do, we decided to do our proof of concept in a more high-level language to start, then transfer that over and implement the real-time aspects once we knew the signal processing sequence was correct. Another unexpected benefit is that the development plan allows us to keep the Vascular Dynamics Lab and our supervisors up to date on what we're working on and when, so they understand where the project's at, what we may need from them, and what we'll be providing to them at the end of the project.

2. In your opinion, what are the advantages and disadvantages of using CI/CD?

The biggest advantage of CI for our project is that it helps us to catch problems early, and as an added benefit, much of it can be automated. Because our project will have so many complex and interconnected aspects in the signal processing pipeline, it will be very important for all of us to make sure our code is up to standards and won't break someone else's. Our plan to have GitHub Actions run our linter and unit tests on every pull request means we find these issues right away instead of letting them propagate and result in unexpected signal outputs. It also keeps our code consistent across the team, which makes reviewing each other's work much easier. Testing and correctness is especially important for our project because it's to be used real-time in a medical research setting, so a good CI/CD pipeline and process means we can better ensure we meet all of these standards that we set for ourselves.

However, one of the biggest disadvantages of CI/CD to our project is that it can't test anything on our hardware. Because much of our system will rely on the Vivid Q ultrasound, our hardware, and the PowerLab 8/30 unit, we will need to test those separately. This means passing CI doesn't

guarantee that the system works on the hardware, and it could give us a false sense of confidence if we're not careful. Another downside is that tests are also only as good as the signals we test against, which may not include the noise and artifacts we'll see from the real ultrasound, though we do aim to use old pre-recorded data to be more consistent with what we'd expect to see in real-time. The Continuous Deployment part of CI/CD also doesn't really apply to our project since we won't be deploying much software live, though it is useful for making our documents available to the lab and our supervisors for review. Overall, we do agree that CI/CD is worth implementing, though we also need to implement other forms of functional testing to ensure our hardware works as intended.

3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Our group has not had any major disagreements with this deliverable, though there has been some debate on what methods, approaches, and technologies to use. At first, some people wanted to implement straight to real-time using hardware, while others wanted to do a prototype first. We decided that before we could make this decision, we needed to get all of the preliminary information and context for the project, so that we would know how best to proceed. From talking with the lab and our supervisors, it became clear that this may be a bit of a complex problem to solve, so we opted to do a high-level prototype first to get the signal processing pipeline down, before transitioning it to real-time. While this was never a major disagreement, it did show us the importance of communicating and making sure everyone is all on the same page before committing to decisions.

Appendix — Team Charter

[borrows from University of Portland Team Charter —SS]

External Goals

[What are your team's external goals for this project? These are not the goals related to the functionality or quality of the project. These are the goals on what the team wishes to achieve with the project. Potential goals are to win a prize at the Capstone EXPO, or to have something to talk about in interviews, or to get an A+, etc. —SS]

Our team aims to develop a working real-time system that provides accurate data and meets the needs of the Vascular Dynamics Lab. In addition to completing the technical objectives of the project, we want to gain experience in digital signal processing, embedded systems, hardware integration, testing, and working with a research laboratory. We also aim to produce a project that we can confidently discuss in future academic and professional settings and to present a clear and functional system at the Capstone Expo.

Attendance

Expectations

[What are your team's expectations regarding meeting attendance (being on time, leaving early, missing meetings, etc.)? —SS]

Team members are expected to attend scheduled team meetings and arrive on time and prepared. If a member needs to arrive late or leave early, they should notify the team in advance whenever possible. Team members are also expected to attend scheduled supervisor meetings unless they have a valid reason for being unavailable.

Acceptable Excuse

[What constitutes an acceptable excuse for missing a meeting or a deadline? What types of excuses will not be considered acceptable? —SS]

Acceptable reasons for missing a meeting or being unable to complete an assigned task include illness, family emergencies, significant academic conflicts, or other circumstances outside the member's control. Consistently forgetting about a meeting or failing to plan ahead will generally not be considered sufficient reasons. Team members should communicate conflicts as early as possible so that the team can adjust plans if necessary.

In Case of Emergency

[What process will team members follow if they have an emergency and cannot attend a team meeting or complete their individual work promised for a team deliverable? —SS]

If an emergency prevents a team member from attending a meeting or completing assigned work, they should notify the team as soon as reasonably possible. The team will redistribute urgent tasks or adjust deadlines when necessary. Once the situation is resolved, the member should communicate the status of their outstanding work and provide an updated estimate for completion.

Accountability and Teamwork

Quality

[What are your team's expectations regarding the quality of team members' preparation for team meetings and the quality of the deliverables that members bring to the team? —SS]

Team members are expected to complete their assigned work to a reasonable standard before bringing it to the team. Code should be tested before being submitted for review, technical claims should be supported by appropriate sources or evidence, and deliverables should be reviewed for errors and consistency. Team members should raise problems as soon as they are identified..

Attitude

[What are your team's expectations regarding team members' ideas, interactions with the team, cooperation, attitudes, and anything else regarding team member contributions? Do you want to introduce a code of conduct? Do you want a conflict resolution plan? Can adopt existing codes of conduct. —SS]

Team members are expected to communicate respectfully and constructively. Disagreements about technical approaches or project decisions should focus on evidence, requirements, and project constraints. Team members should be open to feedback and willing to reconsider their approach when another solution is better supported. Concerns about workload, communication, or team performance should be raised and discussed directly with the team.

Disagreements that cannot be resolved through discussions will go to a vote. If a disagreement persists, the team will document the issue and, when appropriate, seek guidance from the supervisors, TA, or instructor.

Stay on Track

[What methods will be used to keep the team on track? How will your team ensure that members contribute as expected to the team and that the team performs as expected? How will your team reward members who do well and manage members whose performance is below expectations? What are the consequences for someone not contributing their fair share? —SS]

[You may wish to use the project management metrics collected for the TA and instructor for this. —SS]

[You can set target metrics for attendance, commits, etc. What are the consequences if someone doesn't hit their targets? Do they need to bring the coffee to the next team meeting? Does the team need to make an appointment

with their TA, or the instructor? Are there incentives for reaching targets early?
—SS]

The team will use GitHub Issues and GitHub Projects to track tasks, responsibilities, deadlines, and progress. Pull requests, meeting action items, and course deliverables will provide additional checkpoints for monitoring progress.

Each team member is expected to contribute consistently rather than relying on a small number of large contributions near deadlines. The team will not use a fixed minimum number of commits as a measure of contribution, since different tasks may require different amounts of work and may not always involve code. Contribution will instead be considered based on completed tasks, technical work, research, testing, documentation, meeting participation, and other agreed-upon responsibilities.

If a team member repeatedly misses meetings, deadlines, or assigned responsibilities, the team will first discuss the issue with that member and determine how to address it. If the problem continues, the team may involve the TA, supervisor, or instructor to help resolve the situation.

Team members who complete their work early or take on additional responsibilities will help the team maintain progress toward upcoming milestones. The team will recognize these contributions through workload adjustments and by giving members opportunities to take ownership of areas that interest them.

Team Building

[How will you build team cohesion (fun time, group rituals, etc.)? —SS]

The team will make an effort to maintain a positive team environment throughout the project. In addition to regular project meetings, the team may organize informal meals, study sessions, or other social activities when convenient. The team will also recognize major project milestones and celebrate progress toward demonstrations and other important deadlines through celebratory meals.

Decision Making

[How will you make decisions in your group? Consensus? Vote? How will you handle disagreements? —SS]

The team will attempt to reach decisions through discussion and consensus. Technical decisions will be based on project requirements, available evidence, testing results, and practical constraints. When consensus cannot be reached after reasonable discussion, the team will use a majority vote.